ARCHITECTURE & DESIGN GUIDE

Scala 3 + ZIO Todo App

How the backend is built, the Scala and ZIO techniques behind each part, the design decisions and trade-offs, and why ZIO was chosen over the alternatives.

Repository: github.com/digitalDG/scala-zio-fullstack

Live demo: frontend-production-dc1a2c7f.up.railway.app

Code as of commit `aa0f79e` · November 2026

CONTENTS

What's inside

01 **The big picture**
What the app does, its layers, and one request traced end to end.

02 **Effects as values**
`ZIO[R, E, A]`, the type aliases, and chaining with `for`.

03 **Errors vs. defects**
Typed errors, union types, `orDie`, `refineOrDie`, and exhaustive matching.

04 **Dependency injection with ZLayer**
The service pattern, the dependency graph, and swapping in fakes for tests.

05 **Resources, concurrency and time**
Scopes, `Ref`, background fibers, blocking work, and `TestClock`.

06 **HTTP with Tapir**
Endpoints as values, security, validation, and the OpenAPI contract.

07 **The database with Quill**
Queries written in Scala, rows vs. domain types, Postgres features.

08 **Scala 3 language features**
Opaque types, smart constructors, enums, givens, extension methods, and `Validated`.

09 **The testing strategy**
Four kinds of test and what each one proves.

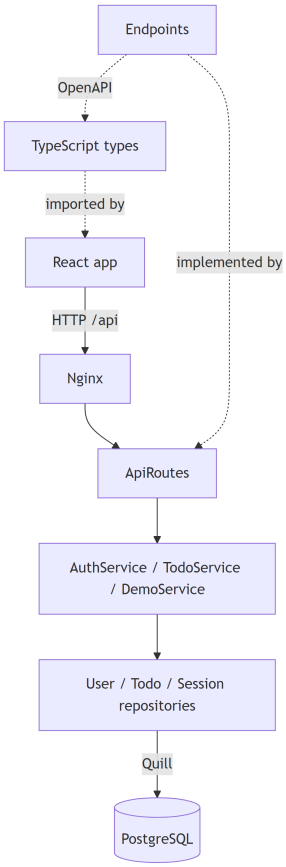
10 **Why ZIO? The alternatives**
Futures, Cats Effect, Akka/Pekko, direct style and Kyo, compared honestly.

A **Appendix: ZIO operators**
Every ZIO operator used in this codebase, in one table.

CHAPTER 1

The big picture

The app is a to-do list. Users register and sign in, and each user's todos are private. A todo has a title, optional notes, a priority, an optional due date and tags. The list can be filtered, searched, sorted and paged. A shared demo account lets visitors try it without registering.



Solid arrows: a request at runtime. Dotted arrows: the API contract, which flows from the Scala endpoint definitions into the frontend's types at build time.

The layers

Each layer talks only to the one below it. That rule makes the code easier to follow and lets each layer be tested on its own.

Layer	Package	Job	Knows about
HTTP	`http`	Describe endpoints, validate requests, turn results and errors into JSON	Tapir, JSON, services
Services	`auth`, `todo`, `demo`	The use cases: sign in, list todos, reset the demo	Repositories, domain
Repositories	`db`	Read and write the database	Quill, SQL, domain
Domain	`domain`	The core types and rules	Nothing else in the app

One request, end to end

Here's what happens on `PUT /api/todos/42`, when a user edits a todo:

1. **Tapir decodes the request** (Chapter 6): the path parameter becomes a `TodoId`, the cookie is read, and the JSON body becomes a `TodoRequest`. Anything malformed gets a 400 before our code runs.
2. **The security step** looks up the session from the cookie and produces an `AuthUser`, or a 401.
3. **Validation** turns the raw request into domain types and collects *every* invalid field (Chapter 8).
4. **The service** runs `TodoService.update`, which only touches todos owned by this user (Chapter 2).
5. **The repository** runs the SQL through Quill (Chapter 7).
6. **The result** becomes a `TodoResponse`. An expected error becomes the right status code; an unexpected one becomes a logged 500 (Chapter 3).

Keep this sequence in mind: each of the following chapters zooms in on one step of it.

CHAPTER 2

Effects as values

This is the most important idea in ZIO. Everything else builds on it.

In ordinary Scala, calling a method *does* the thing: `db.query(...)` hits the database right away. In ZIO, a method returns a **description** of the work, and nothing happens yet. It's like a recipe rather than a cooked meal: you can pass it around, combine it with other recipes, or add "retry three times" or "time out after 2 seconds". Only the ZIO runtime actually runs it, and in this app that happens once, in `Main.run`.

Reading the type

Every effect has the type `ZIO[R, E, A]`:

Parameter	Meaning	In plain words
`R`	Requirements	What it needs to run, such as a database or config
`E`	Error	How it can fail, *as a type*
`A`	Result	What it produces when it succeeds

The code mostly uses shorter aliases for common cases:

Alias	Means	Used for
`IO[E, A]`	`ZIO[Any, E, A]`	Needs nothing, can fail with `E`
`UIO[A]`	`ZIO[Any, Nothing, A]`	Can't fail in an expected way
`Task[A]`	`ZIO[Any, Throwable, A]`	Can fail with any exception
`URIO[R, A]`	`ZIO[R, Nothing, A]`	Needs `R`, can't fail

So `IO[AppError.NotFound, Todo]` reads: "a plan that produces a `Todo`, or fails with `NotFound`, and nothing else". The compiler holds you to that.

In the code: TodoService

`backend/.../todo/TodoService.scala`

```scala
trait TodoService:
  def list(owner: UserId, query: TodoQuery): UIO[Page[Todo]]
  def get(owner: UserId, id: TodoId): IO[AppError.NotFound, Todo]
  def create(owner: UserId, draft: TodoDraft): UIO[Todo]
  def delete(owner: UserId, id: TodoId): IO[AppError.NotFound, Unit]
  // ...

final class Live(repo: TodoRepository) extends TodoService:
  def get(owner: UserId, id: TodoId): IO[AppError.NotFound, Todo] =
    repo.find(owner, id).someOrFail(AppError.todoNotFound(id))
```

Read the signatures first: they're the documentation. `list` can't fail; `get` can fail only with "not found". Then look at `get`. `repo.find` is a `UIO[Option[Todo]]`, a plan that yields "maybe a todo". `.someOrFail(...)` builds a new plan that succeeds with the todo if it's there, and fails with `NotFound` if not. The "maybe missing" case moves out of the *value* and into the *error channel*.

Chaining steps with `for`

backend/.../auth/AuthService.scala

```scala
def register(registration: Registration): IO[AppError.Conflict, AuthResult] =
  for
    hash    <- hasher.hash(registration.password)
    now     <- Clock.instant
    user    <- users.create(NewUser(registration.username, registration.email, hash, now))
    session <- sessions.create(user.id)
  yield AuthResult(user, session)
```

Each `<-` means "run this step, then name its result". The steps run in order, and if one fails, the rest are skipped and the whole effect fails with that error. It's the same `for` syntax you'd use with `Option` or `List`, applied to effects. `users.create` can fail with `Conflict` (username taken), which is why the whole method's type is `IO[AppError.Conflict, …]`: errors flow up through the types.

Notice `Clock.instant`. Even "what time is it?" is an effect here, not a direct call to the system clock. That's what lets tests control time (Chapter 5).

CHAPTER 3

Errors vs. defects

ZIO separates two kinds of failure, and this codebase leans on that separation everywhere:

	Expected errors	Defects
Examples	Username taken, todo not found, wrong password	Database unreachable, a bug, a null
Where they live	The `E` type parameter	Hidden from the type; carried by the runtime
Who handles them	Callers, who must decide what to do	One handler at the edge of the app
Client sees	A specific status and message	A generic 500 with no internal details

Modelling expected errors

backend/.../domain/AppError.scala

```
sealed trait AppError extends Product with Serializable:
  def message: String

object AppError:
  final case class InvalidInput(errors: NonEmptyChunk[FieldError]) extends AppError
  final case class Unauthorized(message: String) extends AppError
  final case class NotFound(message: String) extends AppError
  final case class Conflict(message: String) extends AppError
  final case class TooManyRequests(message: String) extends AppError
  final case class Forbidden(message: String) extends AppError
```

Each case is its own class, so a signature can name exactly which errors it produces, and `sealed` means the compiler knows the complete list.

Union types: listing exactly what can go wrong

backend/.../auth/AuthService.scala

```
def login(
  username: String,
  password: String,
  clientIp: Option[String]
): IO[AppError.Unauthorized | AppError.TooManyRequests, AuthResult]
```

`A | B` is a Scala 3 **union type**: "either an `A` or a `B`". Sign-in fails because the credentials are wrong or because there have been too many attempts, and nothing else. A caller can match on exactly those two cases.

Turning failures into defects: `orDie` and `refineOrDie`

Database calls in Quill return `ZIO[Any, SQLException, A]`. Inputs are validated before they reach the database, so most SQL errors mean something is broken. The repositories turn them into defects with `.orDie`, which is why most repository methods are a clean `UIO`.

Creating a user is the exception: a duplicate username is a normal outcome. `refineOrDie` keeps that one case as a typed error and turns everything else into a defect:

backend/.../db/UserRepository.scala

```
run(users.insertValue(lift(row)).returningGenerated(_.id))
  .map(id => row.copy(id = id).toDomain)
  // A unique-constraint violation is an expected outcome; any other SQL error is a defect.
  .refineOrDie {
    case e: SQLException if e.getSQLState == UniqueViolation =>
      if e.getMessage.contains("users_email_key") then
        AppError.Conflict("That email is already registered")
      else AppError.Conflict("That username is already taken")
  }
```

Handling both at the edge

backend/.../http/ApiRoutes.scala

```
extension [A](effect: IO[AppError, A])
  private def toApi: IO[ErrorResponse, A] =
    effect
      .mapError(ErrorResponse.from)
      .catchAllDefect(defect =>
        ZIO.logErrorCause("Unhandled defect", Cause.die(defect)) *>
          ZIO.fail(ErrorResponse.internal)
      )
```

Every route ends with `.toApi`. Expected errors become API errors via `ErrorResponse.from`. Defects are logged in full on the server, while the client receives only "Internal server error".

Exhaustive matching, and making the compiler strict

backend/.../http/ApiModels.scala

```
def from(error: AppError): ErrorResponse = error match
  case AppError.InvalidInput(errors)     => ErrorResponse(ErrorCode.ValidationFailed, ...)
  case AppError.Unauthorized(message)    => ErrorResponse(ErrorCode.Unauthorized, message)
  case AppError.NotFound(message)        => ErrorResponse(ErrorCode.NotFound, message)
  case AppError.Conflict(message)        => ErrorResponse(ErrorCode.Conflict, message)
  case AppError.TooManyRequests(message) => ErrorResponse(ErrorCode.TooManyRequests, message)
  case AppError.Forbidden(message)       => ErrorResponse(ErrorCode.Forbidden, message)
```

A LESSON FROM BUILDING THIS

Because `AppError` is sealed, the compiler knows when a case is missing from this match. But by default Scala only *warns*; the code still compiles and would crash at runtime. When `Forbidden` was added, it showed up only as a warning. So `build.sbt` now promotes that specific warning (code E029) to an error with `"-Wconf:id=E029:error"`. Now adding an error case without mapping it really won't compile.

CHAPTER 4

Dependency injection with ZLayer

Services need other services: `AuthService` needs a user repository, a password hasher, sessions and a rate limiter. Instead of creating those itself, each service *declares* what it needs, and ZIO builds and connects everything once, at startup.

The service pattern

Every service in the app has the same three parts:

backend/.../todo/TodoService.scala (shape)

```
trait TodoService:                                        // 1. the interface
  def get(owner: UserId, id: TodoId): IO[AppError.NotFound, Todo]

object TodoService:
  final class Live(repo: TodoRepository) extends TodoService:   // 2. the implementation
    def get(owner: UserId, id: TodoId) = ...                     //    dependencies are
                                                                  //    constructor parameters
  val live: URLayer[TodoRepository, TodoService] = ZLayer.derive[Live]   // 3. the layer
```

A **layer** is a recipe for building a service from its dependencies. Read `URLayer[TodoRepository, TodoService]` as "given a `TodoRepository`, I can build a `TodoService`, and building it can't fail". `ZLayer.derive` writes that layer for you, by looking at the constructor's parameters.

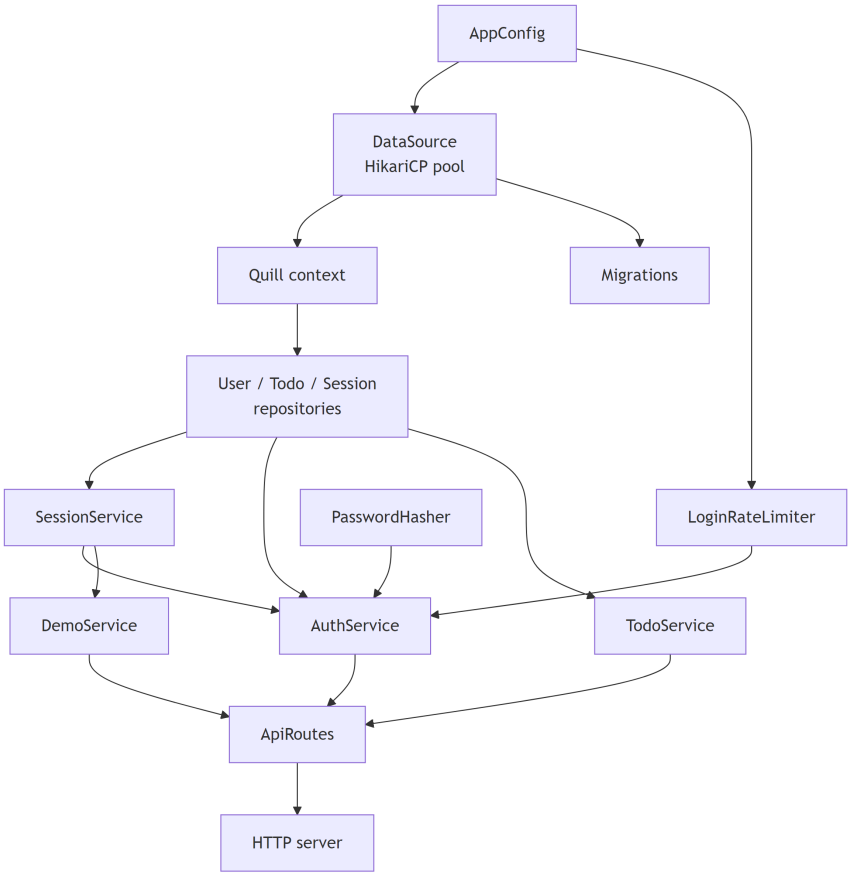
The whole graph, checked at compile time

backend/.../Main.scala

```
def run =
  app.provide(
    AppConfig.layer,
    Database.dataSource,
    Database.quill,
    Migrations.layer,
    UserRepository.postgres,
    TodoRepository.postgres,
    SessionRepository.postgres,
    PasswordHasher.live,
    SessionService.live,
    LoginRateLimiter.live,
    AuthService.live,
    DemoService.live,
    TodoService.live,
    // ... and a few more
  )
```

You list the layers in any order, and `provide` works out the order to build them. If something is missing, such as forgetting `PasswordHasher.live`, you get a **compile error** saying which service is missing and

who needs it. That's the main advantage over runtime DI frameworks, which only fail when the app starts.



The dependency graph that `provide` works out from the layers in `Main` (simplified).

Layers that do work while being built

`AuthService.live` uses `ZLayer.fromZIO` because building it is itself an effect: it hashes a throwaway password at startup. During sign-in, that hash is checked even for usernames that don't exist, so response times don't reveal which accounts are real.

Why this matters for tests

`backend/src/test/.../ApiSpec.scala`

```
private val testLayer: ULayer[ApiRoutes & DemoService] = ZLayer.make[ApiRoutes & DemoService](
  ZLayer.succeed(testConfig),
  InMemoryUserRepository.layer,           // instead of UserRepository.postgres
  InMemoryTodoRepository.layer,           // instead of TodoRepository.postgres
  InMemorySessionRepository.asRepository, // instead of SessionRepository.postgres
  PasswordHasher.fast,                    // bcrypt at its cheapest cost
  SessionService.live,
  LoginRateLimiter.live,
  AuthService.live,
  // ...the rest exactly as in production
)
```

The tests use the real services and real routes. Only the storage is swapped for in-memory versions, so the HTTP tests run in about a second with no database. Swapping is just a matter of passing a different layer.

CHAPTER 5

Resources, concurrency and time

Resources that clean up after themselves

backend/.../db/Database.scala

```
val dataSource: ZLayer[AppConfig, Throwable, DataSource] = ZLayer.scoped {
  for
    cfg  <- ZIO.serviceWith[AppConfig](_.db)
    pool <- ZIO.fromAutoCloseable(ZIO.attemptBlocking {
      val hc = HikariConfig()
      hc.setJdbcUrl(cfg.url)
      // ...
      HikariDataSource(hc)
    })
  yield pool
}
```

A **scope** is a lifetime. `fromAutoCloseable` opens the connection pool and registers "close it" to run when the scope ends. `ZLayer.scoped` ties that scope to the layer, so the pool closes when the app shuts down, even if shutdown is caused by an error. There's no `try/finally` to forget.

Shared state with `Ref`

The sign-in rate limiter counts failures per account and per address. Many requests can arrive at once, so the counters must be updated safely. A `Ref` is ZIO's thread-safe mutable reference: `update` applies a function atomically, so two requests can never both read "4 failures" and both write "5".

backend/.../auth/LoginRateLimiter.scala

```
def recordFailure(keys: List[AttemptKey]): UIO[Unit] =
  Clock.instant.flatMap { now =>
    state.update { windows =>
      // Dropping closed windows here keeps the map from growing without bound.
      val open = windows.filter((_, w) => isOpen(w, now))
      keys.foldLeft(open) { (acc, key) =>
        acc.updated(key, acc.get(key).fold(Window(1, now))(w => w.copy(failures = w.failures + 1)))
      }
    }
  }
```

The function passed to `update` is pure: it takes the old map and returns a new one. A test proves the safety claim by firing 100 failures in parallel with `ZIO.foreachParDiscard`.

Background jobs tied to a lifetime

backend/.../auth/SessionService.scala

```scala
val live: URLayer[SessionRepository & AppConfig, SessionService] = ZLayer.scoped {
  for
    repo   <- ZIO.service[SessionRepository]
    config <- ZIO.service[AppConfig]
    service = Live(repo, config.session.ttlMinutes.minutes, SecureRandom())
    _ <- service.purgeExpired
      .flatMap(n => ZIO.logInfo(s"Deleted $n expired sessions").when(n > 0))
      .catchAllCause(cause => ZIO.logErrorCause("Session cleanup failed", cause))
      .schedule(Schedule.spaced(1.hour))
      .forkScoped
  yield service
}
```

Read it from the inside out: purge expired sessions; log how many; if that fails, log the failure instead of crashing; repeat every hour (`Schedule`); and run it all in the background (`forkScoped`). A **fiber** is ZIO's lightweight thread. `forkScoped` ties it to the scope, so it stops automatically at shutdown. The demo account's 6-hourly reset uses the same pattern.

Blocking work

bcrypt is deliberately slow, and JDBC calls block a thread. ZIO runs ordinary effects on a small pool of threads, so blocking work is wrapped in `ZIO.attemptBlocking`, which moves it to a separate pool where waiting is fine.

Time you can control

`backend/src/test/.../SessionSpec.scala`

```scala
test("sessions expire") {
  for
    fx     <- sessions
    token  <- fx.service.create(fx.user.id)
    _      <- TestClock.adjust(31.minutes)
    result <- fx.service.authenticate(token.value).either
  yield assertTrue(result.isLeft)
}
```

Because the code reads time from `Clock` instead of calling the system clock, the test can jump 31 minutes ahead instantly. No `Thread.sleep`, and no flaky timing.

CHAPTER 6

HTTP with Tapir

Tapir separates *describing* an endpoint from *implementing* it. An endpoint is a plain value that lists its path, inputs, outputs and errors. Behaviour is attached separately.

An endpoint description

backend/.../http/Endpoints.scala

```
private val listParams: EndpointInput[TodoListParams] =
  query[Option[Boolean]]("completed")
    .and(query[Option[Priority]]("priority"))
    .and(query[Option[String]]("tag"))
    .and(query[Option[String]]("q"))
    .and(query[SortField]("sort").default(SortField.CreatedAt))
    .and(query[SortOrder]("order").default(SortOrder.Desc))
    .and(query[Int]("page").default(1).validate(Validator.min(1)))
    .and(query[Int]("pageSize").default(20).validate(Validator.inRange(1, 100)))
    .mapTo[TodoListParams]

val listTodos: Endpoint[Option[String], TodoListParams, ErrorResponse, TodoPageResponse, Any] =
  secured.get
    .in("todos")
    .in(listParams)
    .out(jsonBody[TodoPageResponse])
```

The type `Endpoint[Security, Input, Error, Output, _]` captures the whole contract: the security input (the session cookie), the input (the query parameters), the error type and the output. Validators such as `inRange(1, 100)` mean `?pageSize=500` is rejected before any of our code runs. `mapTo` packs the eight parameters into one case class.

Attaching behaviour

backend/.../http/ApiRoutes.scala

```
private val listTodos: ZServerEndpoint[Any, Any] = Endpoints.listTodos
  .zServerSecurityLogic(authenticate)        // cookie -> AuthUser, or a 401
  .serverLogic { user => params =>           // runs only for signed-in users
    (for
      query <- validate(params.validated)
      page  <- todoService.list(user.id, query)
    yield TodoPageResponse.from(page)).toApi
  }
```

Security logic runs first. The main logic receives the already-authenticated `user`, so it can't accidentally serve a request with no session.

One definition, three products

Because endpoints are values, the same list (`Endpoints.all`) produces:

1. the running server, through `ZioHttpInterpreter`
2. the OpenAPI spec and Swagger UI at `/docs`, through `OpenAPIDocsInterpreter`
3. the frontend's TypeScript types: `sbt openapi` writes the spec, and `openapi-typescript` turns it into types

CI regenerates the spec and types on every push and fails if they differ from what's committed. An API change can't ship without the frontend being updated to match.

Consistent errors, even before our code runs

Malformed JSON or an out-of-range parameter is rejected by Tapir itself. `HttpServer.interceptors` customises those default responses so they use the same `ErrorResponse` JSON as every other error, and the same configuration is shared with the tests.

CHAPTER 7

The database with Quill

Quill lets you write queries as ordinary Scala collection operations, such as `filter`, `sortBy` and `drop`, and translates them into SQL. Column names and types are checked against Scala case classes.

The three Quill words: `quote`, `lift`, `run`

```
private inline def todos = quote(querySchema[TodoRow]("todos"))

def find(owner: UserId, id: TodoId): UIO[Option[Todo]] =
  run(todos.filter(t => t.id == lift(id.value) && t.ownerId == lift(owner.value))).orDie
    .map(_.headOption.map(_.toDomain))
```

- `quote` marks code that becomes SQL, not Scala that runs.
- `lift` brings a runtime value into the query as a bound parameter (`?`), which prevents SQL injection.
- `run` turns the query into an effect: `ZIO[Any, SQLException, List[TodoRow]]`.

Rows vs. domain types

The repository reads `TodoRow`, a plain case class shaped exactly like the table, and converts it with `toDomain` into the richer `Todo`, with opaque IDs, a `Priority` enum and validated titles. The database can store priority as a number, so it sorts correctly, without the domain model caring.

Postgres-specific SQL, and building queries in pieces

backend/.../db/TodoRepository.scala

```
val filtered = quote {
  todos.filter(t =>
    t.ownerId == lift(owner.value) &&
      sql"(${lift(tag)}::text IS NULL OR ${lift(tag)} = ANY(${t.tags}))".asCondition &&
      sql"(${lift(search)}::text IS NULL OR ${t.title} ILIKE ${lift(search)})".asCondition
  )
}

val sorted = query.sort match
  case SortField.DueDate =>
    if asc then quote(filtered.sortBy(t => (t.dueDate, t.id))(Ord(Ord.ascNullsLast, Ord.asc)))
    else quote(filtered.sortBy(t => (t.dueDate, t.id))(Ord(Ord.descNullsLast, Ord.desc)))
  // ... one case per sort field

val items = run(quote(sorted.drop(lift(offset)).take(lift(size))))
val total = run(quote(filtered.size))
(items <*> total).orDie.map((rows, count) => Page(rows.map(_.toDomain), count, query.page))
```

When Quill has no operator for a Postgres feature, such as array membership (`= ANY`) or case-insensitive matching (`ILIKE`), the `sql"..."` interpolator lets you write that fragment directly while still using lifted parameters. Optional filters use the pattern `(param IS NULL OR condition)`, so one query

covers every filter combination. The sort is chosen with an ordinary Scala `match` that builds different quoted queries; sorting by `id` as well makes paging stable. Finally, `<*>` runs the page query and the count query one after another and pairs their results.

WHY THE DATABASE TESTS MATTER

Quill checks the Scala side, but only a real database proves the generated SQL works. `PostgresRepositorySpec` uses Testcontainers to start a throwaway PostgreSQL 18 in Docker, runs the Flyway migrations, and checks every query, including array tags, `ILIKE` with escaped wildcards and nulls-last sorting.

CHAPTER 8

Scala 3 language features

Opaque types: type safety at zero cost

backend/.../domain/Ids.scala

```
opaque type UserId = Long

object UserId:
  def apply(value: Long): UserId = value
  extension (id: UserId) def value: Long = id

opaque type TodoId = Long
// ... the same shape
```

At runtime a `UserId` is just a `Long`, with no wrapper object. But outside this file, the compiler treats `UserId` and `TodoId` as different types, so `todoService.get(todoId, userId)`, with the arguments swapped, doesn't compile. In a codebase full of `Long` IDs, that's a whole class of bugs ruled out.

Smart constructors

backend/.../domain/User.scala

```
opaque type Username = String

object Username:
  private val Pattern = "^[a-z0-9_]{3,32}$".r

  def make(raw: String): Validated[Username] =
    val normalized = raw.trim.toLowerCase
    Validated.check(Pattern.matches(normalized), "username",
      "must be 3-32 characters: letters, digits or underscores")(normalized)

final case class PlainPassword private (value: String):
  override def toString: String = "PlainPassword(***)"
```

`Username.make` is the only way to get a `Username` from user input, so any value of that type has already been validated and normalised. `PlainPassword` goes further: its constructor is `private`, and `toString` is redacted, so a password can't end up in a log by accident.

Enums, and givens that come with them

```
enum Priority:
  case Low, Medium, High

object Priority extends LabeledEnum[Priority]:
  def label(value: Priority): String = value.toString.toLowerCase
  protected def typeName             = "Priority"
```

`LabeledEnum` is a trait whose body defines three `given` instances: a JSON codec, an OpenAPI schema and a query-parameter codec. Because the enum's companion object extends it, those givens are part of the companion. Scala finds givens in companions automatically, so anywhere a `JsonCodec[Priority]` is needed, it's found with no imports. One line gives the enum everything it needs to travel over HTTP, and the frontend receives it as the TypeScript union `"low" | "medium" | "high"`.

Derivation

`final case class LoginRequest(username: String, password: String) derives JsonCodec, Schema` asks the compiler to generate the JSON codec and API schema from the case class's fields. There's nothing to keep in sync by hand.

Validated: collecting every error at once

backend/.../domain/Validated.scala and http/ApiModels.scala

```
type Validated[+A] = Either[NonEmptyChunk[FieldError], A]

extension [A](self: Validated[A])
  def zipPar[B](that: Validated[B])(using z: Zippable[A, B]): Validated[z.Out] =
    (self, that) match
      case (Right(a), Right(b)) => Right(z.zip(a, b))
      case (Left(e1), Left(e2)) => Left(e1 ++ e2)     // keep the errors from both sides
      case (Left(e), Right(_))  => Left(e)
      case (Right(_), Left(e))  => Left(e)

def validated: Validated[Registration] =
  Username.make(username)
    .zipPar(Email.make(email))
    .zipPar(PlainPassword.make(password))
    .map(Registration.apply)
```

A plain `Either` stops at the first error. `zipPar` keeps the errors from both sides, so a form with three bad fields reports all three. Two more Scala features are at work here:

- **Extension methods** add `zipPar` to a type we don't own (`Either`).
- **A dependent type**: `z.Out` is a type that depends on the `Zippable` value found for `A` and `B`. ZIO's `Zippable` flattens tuples, so three validations give `(Username, Email, PlainPassword)` rather than `((Username, Email), PlainPassword)`, and `Registration.apply` takes them directly.

CHAPTER 9

The testing strategy

There are four kinds of test, each proving something the others can't. The backend has 56 tests and the frontend 18.

Kind	Example	Proves	Speed
Unit	`ValidationSpec`, `SessionSpec`, `ConfigSpec`	Rules, sessions, the rate limiter and config parsing, in isolation	Milliseconds
HTTP API	`ApiSpec`	Routing, JSON, validation, auth and error mapping, through the real routes with in-memory storage	About 1 second
Database	`PostgresRepositorySpec`	The real SQL against PostgreSQL 18 in Docker	A few seconds
Frontend	Vitest + Testing Library	Components, URL filters, session handling	Seconds

Techniques worth noticing

- **Layer swapping** (Chapter 4): the API tests are the production wiring with in-memory repositories.
- `TestClock` (Chapter 5): session expiry and lockout windows are tested without waiting.
- **Tapir's stub interpreter**: `ApiSpec` sends real HTTP requests through the real routes without opening a network port.
- **Skipping cleanly**: the database tests check whether Docker is available and mark themselves ignored if not, so `sbt test` still works on a laptop without Docker. In CI, Docker is always there, so they always run.
- **Testing behaviour, not implementation**: tests like "users can't see or change each other's todos" and "the demo account can't be locked out" describe what the app promises, not how it's coded.

A TEST THAT CAUGHT A REAL BUG

While the demo account was being built, two API tests failed: signing in to the demo before the first scheduled reset created the user but no sample todos. Production happened to run the reset first, so the bug would never have shown there, but sign-in depended on that order without saying so. The fix was to make sign-in run a full reset when the account doesn't exist yet.

CHAPTER 10

Why ZIO? The alternatives

Scala has several mature ways to write a backend. Here are the main options, and why this project uses ZIO, with the same small function in each style where it helps.

1. Plain `Future`s

Scala's standard library. A `Future` starts running as soon as it's created (it's *eager*), its error type is always `Throwable`, it can't be cancelled, and every call needs an `ExecutionContext`. It's fine for small programs, but resource safety, timeouts, retries and testing time all have to be built by hand. Frameworks like Play still use it.

2. Cats Effect and the Typelevel stack

ZIO's closest peer, with the same core idea: effects as lazy values, fibers, resource safety. The ecosystem includes http4s (HTTP), Doobie or Skunk (database), fs2 (streaming) and Circe (JSON). The main differences:

- **Errors:** `IO[A]` fails with `Throwable`. Typed errors are added with `EitherT` or a library, rather than being built in.
- **Dependencies:** there's no built-in DI. Services are passed through constructors and composed with `Resource`.
- **Style:** code is often written in "tagless final" style, generic over the effect type `F[_]`, which is very flexible but more abstract.

```
// Cats Effect, tagless final style
trait TodoService[F[_]]:
  def get(owner: UserId, id: TodoId): F[Either[NotFound, Todo]]

// ZIO: the error is part of the effect type itself
trait TodoService:
  def get(owner: UserId, id: TodoId): IO[AppError.NotFound, Todo]
```

3. Akka, and its fork Apache Pekko

Built around the **actor model**: independent actors holding private state and exchanging messages. It's excellent for distributed, stateful systems, such as clustering, event sourcing and long-lived workflows. For a request/response API like this one, actors add ceremony without much benefit. Akka moved to a commercial licence (BSL) in 2022; Apache Pekko is the community fork under the Apache licence.

4. Direct style on virtual threads

Since Java 21, virtual threads make blocking cheap, so plain "call it and wait" code can scale. Libraries like SoftwareMill's **Ox** add structured concurrency on top. The code looks like ordinary Scala, which

makes this the easiest option to learn. The trade-off: effects aren't values, so retries, timeouts and resource handling are more manual, and the ecosystem is younger.

```
// Direct style: no effect type at all
def get(owner: UserId, id: TodoId): Either[NotFound, Todo] =
  repo.find(owner, id).toRight(NotFound(s"Todo $id not found"))
```

5. Kyo

A newer library built on **algebraic effects**, where a type lists every effect a computation uses. It's very expressive and interesting, but young, with a small community. Worth watching, but a riskier foundation for a production service today.

Side by side

	ZIO 2	Cats Effect	Akka / Pekko	Direct style (Ox)	Future
Typed errors	Built in (`E`)	Via `EitherT` / libraries	Messages / exceptions	`Either` / exceptions	`Throwable` only
Dependency injection	Built in (`ZLayer`)	Manual / `Resource`	Manual	Manual	Manual
Resource safety	`Scope`	`Resource`	Actor lifecycle	Structured scopes	By hand
Testable time	`TestClock`	`TestControl`	TestKit	Limited	None
Learning curve	Steep but guided	Steepest (abstractions)	Different model	Gentlest	Gentle
Ecosystem	Large, consistent	Large, mature	Mature	Young	Standard library

Why ZIO for this project

1. **Everything is in one ecosystem.** ZIO HTTP, zio-json, zio-config, zio-logging, ZIO Test and Quill's ZIO module are designed to fit together, so there are few adapters between libraries and one set of conventions throughout.
2. **Typed errors are built in.** The error-handling design in Chapter 3, with precise per-method errors, union types and defects kept separate, is ZIO's default, not an add-on.
3. **Dependency injection comes with compile-time checks.** Missing services are compile errors, and tests swap layers trivially.
4. **The test kit is strong.** `TestClock`, layer-based fakes and `assertTrue` make the tests in Chapter 9 short.
5. **It's concrete.** ZIO code uses one concrete effect type rather than abstracting over `F[_]`, which keeps signatures and compiler errors easier to read.

The honest downsides

- **Large API surface.** There are many ways to do the same thing (`orDie`, `refineOrDie`, `catchSome`, `mapError`...), and learning which to use takes time.
- **Signatures get busy.** `ZIO[Migrations & DemoService & ApiRoutes & AppConfig & Server, Throwable, Nothing]` is a mouthful.
- **Ecosystem choice.** ZIO and Typelevel libraries don't mix naturally; interop modules exist, but you mostly pick one side.
- **It can be more than a small app needs.** For a small CRUD app, direct style on virtual threads would be shorter. ZIO earns its keep as concurrency, resources and failure modes grow, which is exactly what the planned real-time streaming feature will add.

THE DECISION IN SHORT

ZIO was chosen for built-in typed errors, compile-checked dependency wiring and a single consistent ecosystem. Cats Effect would give similar guarantees in a more abstract style, and for a service this small, direct style on virtual threads would also be reasonable. ZIO pays off most as concurrency and resource handling grow, which the planned real-time features will add.

APPENDIX

Cheat sheet: ZIO operators in this codebase

Operator	What it does	Where to see it
`ZIO.succeed(a)` / `ZIO.fail(e)`	An effect that succeeds with `a` / fails with `e`	`ApiRoutes`
`map` / `flatMap` / `for`	Transform a result / chain the next step	Everywhere
`mapError`	Transform the error	`toApi`
`someOrFail(e)`	Turn `Option` results into success or a typed error	`TodoService.get`
`someOrElseZIO(eff)`	Use the value, or run `eff` if it's `None`	`DemoService.signIn`
`orDie`	Make every error a defect	Repositories
`refineOrDie { case … }`	Keep some errors typed, make the rest defects	`UserRepository.create`
`catchAllDefect` / `catchAllCause`	Handle defects / handle everything	`toApi`, background jobs
`tapError`	Do something on failure without handling it	`AuthService.login`
`either`	Move the error into the result as an `Either`	Tests
`when(cond)` / `unless(cond)`	Run only if the condition holds / doesn't	`DemoService`
`<*>` / `*>`	Run both and pair results / run both and keep the second	`TodoRepository.list`, `toApi`
`ZIO.foreachDiscard` / `foreachParDiscard`	Run an effect per element, in order / in parallel	`DemoService`, `SessionSpec`
`attemptBlocking`	Run blocking code on the blocking pool, catching exceptions	`PasswordHasher`, `Database`
`Ref.make` / `update` / `modify`	Thread-safe mutable state	`LoginRateLimiter`, in-memory repos
`schedule(Schedule.spaced(d))`	Repeat after each interval	`SessionService`, `DemoService`
`forkScoped`	Run in a background fiber that ends with the scope	`SessionService.live`
`ZIO.scoped` / `ZLayer.scoped`	Open a lifetime for resources and fibers	`Main`, `Database`

`ZIO.fromAutoCloseable`	Acquire a resource, closing it when the scope ends	`Database`
`ZIO.service[S]` / `serviceWith`	Get a service from the environment	Layers, `Main`
`ZLayer.derive` / `fromZIO` / `succeed`	Build a layer from a constructor / an effect / a value	Every service
`provide(...)` / `ZLayer.make`	Wire layers, checked at compile time	`Main`, `ApiSpec`
`Clock.instant` / `TestClock.adjust`	Read time / move test time forward	Services, tests